

Containerization basics

COMPLETE LEARNING GUIDE

What You'll Learn

Containers solve the 'it works on my machine' problem by packaging an application together with its exact runtime environment — dependencies, system libraries, configuration defaults — into a single portable unit that runs identically anywhere a container runtime is available. This guide covers the distinction between an image (a static blueprint) and a container (a running instance of it), writing a Dockerfile, the layer caching model that determines build speed, multi-stage builds for lean production images, Docker Compose for coordinating multiple services (an app plus its database) locally, persisting data with volumes, and the basics of how containers communicate over a network.

1. Images vs. Containers

A Docker image is a read-only, layered blueprint — application code, a runtime, installed dependencies, and configuration, all baked into a static artifact built once from a Dockerfile. A container is a running (or stopped) instance of that image, with its own writable layer on top and its own isolated process, filesystem view, and network namespace. The relationship mirrors a class and an object in object-oriented programming: many containers can be spun up from the exact same image, each an independent running instance.

2. Writing a Dockerfile

A Dockerfile is a sequence of instructions that Docker executes in order to build an image: FROM specifies the base image to start from, WORKDIR sets the working directory inside the image, COPY brings files from the build context into the image, RUN executes a command during the build (like installing dependencies), and CMD specifies the default command to run when a container starts from the image. Each instruction creates a new layer, and layers are cached — which is why the order of instructions has a direct, significant impact on build speed.

DOCKERFILE

```
FROM node:20-alpine
WORKDIR /app

# Copy only dependency manifests first so npm install is cached
# unless package.json/package-lock.json actually change
COPY package*.json ./
RUN npm ci --omit=dev

# Now copy the rest of the source code
COPY . .

EXPOSE 3000
CMD ["node", "server.js"]
```

3. Layer Caching and Build Optimization

Docker caches each layer and reuses it on subsequent builds as long as nothing that affects that layer has changed — but the moment one layer's inputs change, every layer after it must be rebuilt, even if their own inputs didn't change. This is exactly why copying `package.json` and running `npm install` before copying the rest of the application source (as shown above) matters: editing a source file doesn't invalidate the dependency-install layer, so rebuilds after a routine code change skip reinstalling every dependency and are dramatically faster.

4. Multi-Stage Builds

A multi-stage build uses multiple `FROM` instructions in one Dockerfile, where an early stage can include heavy build tooling (a full Node.js toolchain, a TypeScript compiler) that's needed only to produce build artifacts, and a later, final stage copies just those artifacts into a much smaller runtime base image — discarding the build tools entirely from the final image. This routinely shrinks a production image from several hundred megabytes down to tens of megabytes, which matters directly for deploy speed, storage cost, and reduced attack surface.

DOCKERFILE

```
# Stage 1: build
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: run — only the compiled output ships, not the build tooling
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
CMD ["node", "dist/server.js"]
```

5. Docker Compose for Multi-Service Apps

A real full-stack app is rarely just one container — it's typically an API server, a database, and maybe a cache, all needing to run together and talk to each other. Docker Compose defines all of these services declaratively in a single `docker-compose.yml`, and a single `docker compose up` command builds/starts every service, wires up a shared network so they can reach each other by service name, and manages their lifecycle together — replacing what would otherwise be a manual sequence of individual `docker run` commands with the correct flags every time.

YAML

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://postgres:postgres@db:5432/app
    depends_on:
      - db

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: postgres
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

6. Volumes and Persistent Data

A container's writable layer is ephemeral by default — data written inside it disappears the moment the container is removed, which is a serious problem for anything stateful like a database. A volume is a persistence mechanism managed by Docker that lives outside any single container's lifecycle: it can be mounted into a container at a specific path, survive that container being stopped, removed, and recreated, and even be shared between multiple containers. Any container running a database, as in the Compose example above, needs its data directory mounted to a volume, or every restart silently wipes all data.

7. Container Networking Basics

By default, Docker Compose puts all services defined in the same file on a shared network where each can reach the others using the service name as a hostname (the `api` service connects to `db:5432`, not `localhost:5432` or a hardcoded IP) — Docker's internal DNS resolves the service name automatically. Ports need to be explicitly published (the `'3000:3000'` mapping) only for services that need to be reachable from outside the Docker network entirely, like the API server being reachable from a browser on the host machine; internal service-to-service communication doesn't require publishing any ports.

Common Pitfalls

- Copying the entire project directory before installing dependencies, busting the dependency-install layer's cache on every single code change.
- Running the containerized application as the root user by default instead of creating and switching to a non-root user, unnecessarily widening the container's attack surface.
- Shipping a bloated production image because a multi-stage build wasn't used, leaving build tools and dev dependencies in the final image.
- Storing a database's data files only inside the container's writable layer with no volume, silently losing all data on the next container restart or recreation.
- Not pinning a specific base image version (using `node:latest` instead of `node:20-alpine`), causing builds to be inconsistent across time as the 'latest' tag moves.
- Trying to reach another Compose service via `localhost` instead of its service name, since each container has its own isolated network namespace.
- Exposing more ports than actually need to be reachable from outside the container network, unnecessarily widening the attack surface.

Key Takeaways

- An image is a static blueprint; a container is a running instance of it — many containers can share one image.
- Order Dockerfile instructions so rarely-changing steps (dependency installs) come before frequently-changing ones (copying source code) to maximize layer cache hits.
- Multi-stage builds keep heavy build tooling out of the final production image, often shrinking it dramatically.
- Docker Compose coordinates multiple services locally, giving them a shared network where they reach each other by service name.
- Any stateful service (a database) needs its data directory mounted to a volume, or restarts will silently wipe its data.
- Service-to-service communication inside Compose uses service names as hostnames, not `localhost` or published host ports.

Learn More

Docker Docs — Dockerfile Reference

<https://docs.docker.com/reference/dockerfile/>

Docker Docs — Multi-Stage Builds

<https://docs.docker.com/build/building/multi-stage/>

Docker Docs — Compose File Reference

<https://docs.docker.com/reference/compose-file/>